



Published in Image Processing On Line on YYYY-MM-DD.
 Submitted on YYYY-MM-DD, accepted on YYYY-MM-DD.
 ISSN 2105-1232 © YYYY IPOL & the authors CC-BY-NC-SA
 This article is available online with supplementary materials,
 software, datasets and online demo at
<https://doi.org/10.5201/ipol>

A Bidirectional Optimization for Sphere Traced Ray-marching

Diego Fonseca¹

¹ Department of Mathematics, NVCC, Manassas Campus

PREPRINT August 12, 2025

1 Introduction

Ray-marching is a three-dimensional rendering technique in computer graphics where a ray traverses through a scene. As the name suggests, we place a ray at the origin in a set direction. However, the ray-marching algorithm comes into play when we estimate the increase in our magnitude every iteration by using a specific function. In this paper, we focus solely on Ray-marching using Sphere tracing, a type of function that approximates the nearest distance from all objects in our scene to our ray and adds the output of our function to the ray's magnitude.

We present an open-source graphics application that focuses solely on rendering using ray-marching, with available sample configurations [5], to render various types of 3-D scenes in real time, featuring a wide range of virtual navigation. Additionally, the application allows one to test GPU runtime performance and measure of time it takes to render each frame.

The findings of this paper rely on concepts such as *idempotent semirings* $(\mathbb{K}, \oplus, \odot, \bar{0}, \bar{1})$ and the *Riemann Rearrangement Theorem*. The following are fundamental semiring concepts as defined here [10]. We will only be discussing *real tropical semirings* $(\mathbb{R} \cup \{\infty\}, \min, +, \infty, 0)$ and *boolean semirings* $(\{0, 1\}, \vee, \wedge, 0, 1)$, as these make up a good portion of the framework behind ray-marching and its bidirectional counterpart.

2 Related Work

While several enhancements to classical sphere tracing have been proposed in the past, most notably by Keinert et al. [8], these approaches focus on a unidirectional traversal. Their contributions include over-relaxation of step sizes, screen-space-aware intersection selection, dynamic epsilon offsets, and discontinuity correction via fixed-point iteration. However, all techniques operate within the traditional forward-marching paradigm and do not restructure the traversal model itself.

In contrast, our method introduces a bidirectional ray-marching formulation grounded in rearranged Riemann integration, stemming from the Riemann rearrangement theorem. By tracing from both endpoints and detecting convergence internally, our approach offers a topologically distinct mechanism for handling SDF accumulation, particularly effective in scenes with non-Euclidean topology, high divergence fields, or infinite-valued SDFs.

3 Logic Behind Ray-Marching

Though ray-marching is still a relatively recently developed technique introduced back in 1989 as a way to render 3-D fractals [6], it follows the same purpose as ray-tracing in the regard that for each pixel on our screen, we shoot a ray from the origin position of our virtual camera to determine what color of the pixel corresponding to said ray should be. Unlike ray-tracing, however, the objects in the scene hit by our ray are not defined via polygons but through *signed distance functions* [2, p. 10]. Here we present the following.

Definition 1. Let $S = \{x | x : \mathbb{R}^3 \rightarrow \mathbb{R}\}$ such that it represent as a set of implicit surfaces in the scene. To begin, let's defined our ray as

$$r(t) = r_o + (t)r_d \quad (1)$$

where each point $p \in r(t)$ represents how far our ray has traversed, therefore, plugging our ray equation $r : \mathbb{R}_{t \geq 0} \rightarrow \mathbb{R}^3$ into the signed distance function $\text{SDF} : \mathbb{R}^3 \times S \rightarrow \mathbb{R}$ the gives the nearest implicit surface from p from S . For demonstration, let our

$$\text{SDF}(p, S) = ||p - c|| - R \quad (2)$$

where SDF returns a sphere the radius of R centered at c as shown in Figure 1(a). This interpretation underlies the sphere tracing algorithm [7], where each ray-marching step advances by the distance given by $\text{SDF}(p, S)$, ensuring that the surface is not overshoot. The method is both robust and efficient for rendering implicit surfaces, particularly when SDF is differentiable.

3.0.1 Drawing with Ray-marching

Given a signed distance function SDF , we assume the ray origin p_0 lies outside the surface. The value $\text{SDF}(p_0)$ represents the minimum distance to the surface and thus determines the first step size along the ray. At each subsequent iteration, we advance the ray by the signed distance at the current position, p_i , and check for termination under one of the following conditions:

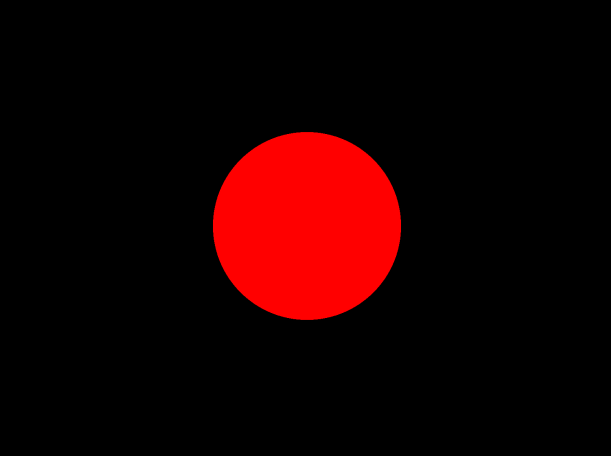
- The maximum number of iterations has been reached,
- The accumulated ray length exceeds a predefined maximum distance,
- The signed distance evaluates to a value less than a small threshold ε (indicating surface proximity).

If the third condition is met, we declare that the ray has intersected the surface at position p_n and proceed to compute the pixel's color based on scene data. If no condition is satisfied, we update the ray position to $p_{n+1} = p_n + \text{SDF}(p_n, S) \cdot r_d$ and repeat the process.

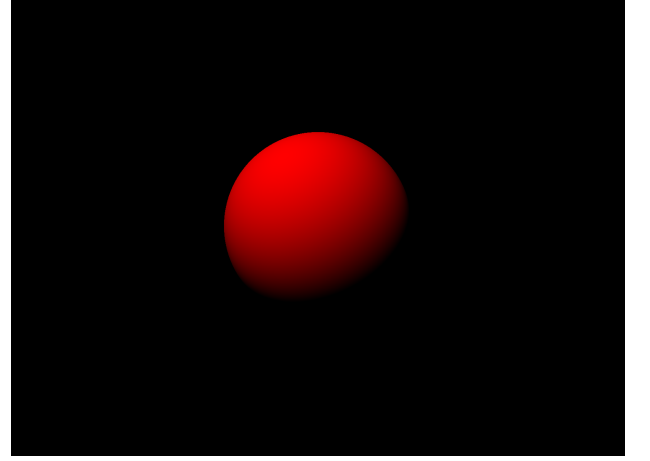
Once an intersection is found, the pixel color is determined using lighting and texturing models. For lighting, the surface normal at the intersection point is required. This normal vector can be approximated by the gradient of the signed distance function:

$$\hat{n}(p_f) := \hat{\nabla} \text{SDF}(p_f, S) \quad (3)$$

The resulting normal is used in a lighting model (e.g., Lambertian or Phong) to compute illumination based on the dot product between the normalized surface normal and the light direction [3]. As shown in Figure 1(b), this enables shading of implicitly defined surfaces.



(a) Ray-marched sphere with no extra rendering other than drawing a red pixel if the ray intersected with the scene.



(b) Ray-marched sphere with basic diffuse lighting applied on its implicit surface.

Figure 1: Comparison of ray-marched spheres without and with diffuse lighting.

3.1 Tropical Semirings for Safest Traversal

The general purpose of this subsection is to explain the logic behind rendering multiple objects in our scene using ray-marching. We are specifically discussing about tropical semirings because they are the traditional structure underlying many shortest-path algorithms [10], plus, it provides a new perspective on how a *signed distance function* can be presented mathematically. Thus, as we progress, we will introduce concepts that will provide a framework for the remainder of the paper.

Definition 2. To start off, let's discuss on what tropical semirings are in the first place. First off, let's introduce tropical addition and multiplication in action using $x, y \in \mathbb{R}$ such as

$$x \oplus y := \min(x, y) \quad \text{and} \quad x \odot y := x + y \quad (4)$$

where extending by introducing z to where $z \in \mathbb{R}$ results in

$$x \oplus y \oplus z := \min(x, y, z) \quad \text{and} \quad x \odot y \odot z := x + y + z \quad (5)$$

Hence, the tropical operations are similar to regular operations [12, p. 163].

Lemma 1. Referring to definition 1, let $p_n = r_o + t_n r_d$ such that r_o is the ray's origin and r_d is the ray's unit direction and let S represent the set of implicit surfaces are in the scene. Then our signed distance function can be expressed as an infimum over a tropical addition, such that

$$SDF(p_n, S) = \bigoplus_{s \in S} s(p_n) \quad (6)$$

where $SDF(p_n, S)$ returns the shortest distance from p_n to an implicit surface in our set S .

Proof. From our formula

$$\bigoplus_{s \in S} s(p_n)$$

we express express tropical summation over a set of SDFs evaluated at point p_n . By definition 2, the tropical addition operator $\oplus$ is the minimum between two operands, where $\bigoplus$ is the infimum operator of tropical addition.

Since $\text{SDF}(p_n, S)$ is defined as the accumulation of all object-wise evaluations using $\oplus$, then

$$\text{SDF}(p_n, S) = \bigoplus_{s \in S} s(p_n)$$

by definition is the tropical addition over set S in such we find the minimum from each $s(p_n)$, the distance of p_n to surface. Therefore, after the end of our series, it returns the shortest distance from p_n to an implicit surface in our set S . $\square$

Lemma 2. Let $\Delta t_n = \text{SDF}(p_n, S)$ and $t_{n+1} = \text{SDF}(p_n, S) + t_n$. Confirm that

$$\Delta t_n = t_{n+1} - t_n \quad (7)$$

where Δt being the difference between two values still holds.

Proof. Define

$$t_{n+1} - t_n = \text{SDF}(p_n, S) + t_n - t_n = \text{SDF}(p_n, S) = \Delta t_n$$

$\therefore \Delta t_n = t_{n+1} - t_n$ holds. $\square$

Proposition 1. Let

$$t_f = \int_S g(r(t)) dt \quad (8)$$

where it can be represented in tropical semirings $(\mathbb{R} \cup \{\infty\}, \min, +, \infty, 0)$ such that

$$t_f \approx \bigodot_{n=0}^N (g(p_n) \cdot \bigoplus_{s \in S} s(p_n)) \quad (9)$$

Proof. Given that t_f is a standard integral, we can approximate it as a standard Riemann sum

$$t_f = \int_S g(r(t)) dt \approx \sum_{n=0}^N g(p_n) \Delta t_n$$

where $p_n = r(t_n)$ represents the discrete points along the ray. Applying Lemma 2

$$t_f \approx \sum_{n=0}^N (g(p_n) \cdot \text{SDF}(p_n, S))$$

Finally, applying Lemma 1:

$$t_f \approx \sum_{n=0}^N (g(p_n) \cdot \bigoplus_{s \in S} s(p_n))$$

In ray-marching, we're interested in the accumulation minimum distances rather than their arithmetic sum. The standard summation $\sum$ in the Riemann sum context represents the cumulative marching distance, but in tropical algebra, this accumulation is represented by tropical multiplication $\odot$. From Definition 2, we can rewrite the $\sum$ operator as $\odot$. Therefore,

$$t_f \approx \bigodot_{n=0}^N (g(p_n) \cdot \bigoplus_{s \in S} s(p_n))$$

$\square$

Algorithm 1 Ray-marching with Tropical SDF(Modified algorithm of [7])

```

1:  $t \leftarrow 0$ 
2:  $p \leftarrow r(t) = r_0 + t \cdot r_d$ 
3: for  $n = 0$  to  $N$  do
4:    $d \leftarrow \text{SDF}(p, S)$   $\triangleright$  For simplicity sake, set  $g(p) = 1$ 
5:   if  $d < \varepsilon$  then
6:     return hit at  $p$ 
7:   end if
8:   if  $d > D_{\max}$  then
9:     return no hit (ray escaped)
10:  end if
11:   $t \leftarrow t + d$ 
12:   $p \leftarrow r(t)$ 
13: end for
14: return no hit (ray escaped)

```

Formalizing the ray-marching process as described in Propositions 1 using the discrete algorithm. At each iteration, the ray advances by the signed distance at the current point, as defined in the tropical semiring. This process is shown in Algorithm 1 with a visual example of each iteration shown in Figure 2.

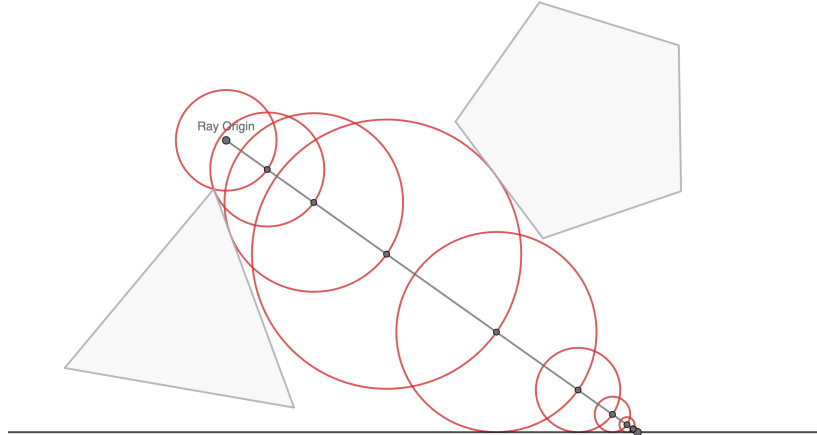


Figure 2: Sphere-Tracing in action. The radius of each circle is Δt_n for each center point p_n . In this specific example, our ray is converging onto a flat surface. Once our radius reaches $\Delta t_f < \varepsilon$, then we’ve hit the surface, thus making our total traversal t_f .

3.2 Ray-marching vs. Ray-tracing

Ray-tracing and ray-marching are both very similar in the way that they share the same purpose: have a ray intersect an implicit surface. However, unlike ray-marching, ray tracing involves using a direct intersection between the ray and polygons of a mesh. When it comes to ray-tracing a mesh with thousands, millions, or even billions of individual triangles, we rely on optimization techniques that are specific to ray-tracing. One of them is the Bounding Volume Hierarchy (BVH), a method that subdivides triangles into a tree-based algorithm, where the largest bounded volumes are at the top branch [9]. The purpose of this is to determine the general area where our ray intersects,

allowing us to proceed properly down each branch where our bounding volume becomes smaller and smaller, narrowing the search to a small set of triangles. This gives an advantage over ray-marching, as the ray already knows which triangle and volume it has to intersect without requiring iterations, technically.

However, this is not to say that ray-tracing comes without its drawbacks; otherwise, we wouldn't be discussing ray-marching as an alternative. Since the original intent for ray-marching was to render 3D fractals more efficiently [6], ray-marching consumes orders of magnitude less memory. Based on the data provided here [11, Table 2], a Menger Sponge uses 1 MB with ray-marching vs. 329 MB of V-RAM with mesh boundary representation, while the FPS to render the Menger Sponge is nearly double with ray-marching.

3.3 The Drawbacks of Ray-marching

Ray-marching, especially sphere tracing, is not used nearly as frequently as ray-tracing. Why is that the case? This is because ray-marching can be very computationally expensive for rendering scenery [13], particularly for Euclidean, voxel-based, and polygonal objects, as it requires multiple iterations to draw a single pixel in which ray-tracing can do in some cases can do just one iteration, as mentioned in the previous section. This is why optimizations are significant for ray-marching, as they are necessary in cases where they can become significantly less expensive.

That's why, in this paper, we will explore a type of optimization for ray-marching that allows us to determine ahead of time whether our ray will diverge or converge, thereby avoiding the waste of performance by computing a pointless operation. This method has only been tested with regular sphere tracing and sphere tracing when rendering meshes through signed distance functions.

4 Integration Behind Bi-directional Ray-marching

If $\Delta t_f \in [0, \varepsilon)$, if our $\Delta t_f < 0$, we exit immediately. This means that we don't necessarily traverse through any negative values in our series, as we are only focusing on sphere tracing when considering ray-marching. This means that in our series, once a negative number appears, we stop the series there, thus making it technically a non-negative series given our particular case. This means our series only contains values in the range $[\varepsilon, D_{\max})$, where $D_{\max} \in \mathbb{R}_{>\varepsilon}$ is the maximum distance beyond which we consider it diverging.

Now, since our series is no longer negative, we can prove that if our series is rearranged, it will not only converge depending on whether our original series converged, but it will also converge at the same value as our original series. This means that our t_J , the total of the rearranged sequence of our original sequence until $\Delta t_f < \varepsilon$, is equal to t_f . Let's prove this.

Definition 3. Let $\sum a_n$ be a series with $a_n \in \mathbb{R}_{\geq 0}$. If $\sum a_n$ converges to M , then for any permutation $\sigma : \mathbb{N} \rightarrow \mathbb{N}$, the rearranged series $\sum a_{\sigma(n)}$ also converges to M . This holds because, for non-negative series, rearrangement preserves the sum (see, e.g., [1, Theorem 2.2]).

4.1 Modification Process of a Series

Let B_N be a modified series of A_N where $A_N = \sum_{n=0}^N f(x_n)$, where $\forall f(x_n) \in \mathbb{R}_{\geq 0}$. In this sub-section, we will go through the process of modifying our A_N . We will also heavily rely on the previous subsections when it comes to properly designing our new series.

Definition 4. Let our Boolean semiring $\beta = (\{0, 1\}, \vee, \wedge, 0, 1)$ be a bounding semiring such that $1 \wedge 1 = 1 \vee 0 = 1$ and $1 \wedge 0 = 1 \vee 1 = 0$. Where $\vee$ denotes XOR, and $\wedge$ denotes AND. Since boolean

semirings can only work with the set $0, 1$, let's express a function that gets us the least significant bit $LSB : \mathbb{N}_0 \rightarrow \{0, 1\}$ where $LSB(x) = x \bmod 2$.

Definition 5. Stemming from Definition 4, let $\gamma(x) : \mathbb{N}_0 \rightarrow \{0, 1\}$ be a function that depends on the input and returns the opposite of LSB . Therefore, $\gamma(x) = LSB(x) \vee 1$.

Definition 6. For sets I and J used for Lemma 3, and further proofs, define:

$$\mathcal{P} \subseteq I \times J \text{ such that } \bigcup_{(i,j) \in \mathcal{P}} \{i, j\} = I \cup J \text{ and } \forall (i, j) \neq (i', j') \in \mathcal{P}, i \neq i' \wedge j \neq j' \quad (10)$$

such that no two ordered pairs in $\mathcal{P}$ share the same first or second coordinate. Set $\mathcal{P}$ is better defined as a sequence of ordered pairs, implying that $\mathcal{P}$ when applied to an application-al use case, P must be interpreted as:

$$\mathcal{P} = \{(i_0, j_0), (i_1, j_1), \dots, (i_n, j_n)\} \quad (11)$$

in simpler terms, an ordered tuple, but for the sake of being universal, let's express $\mathcal{P}$ as a set unless needed to be ordered.

Lemma 3. Let $A_N = \sum_{n=0}^N f(x_n)$ where $f(x_n) \in \mathbb{R}_{\geq 0}$. Any number in general. The series A_N can be expressed in a way where

$$\Gamma(i) = \begin{cases} f(x_0), & \text{if } \gamma(i) = 1 \\ 0, & \text{otherwise} \end{cases} \quad (12)$$

Given $\gamma(N) \leq N$, prove that

$$A_N = \sum_{n=0}^N f(x_n) = \Gamma(N) + \sum_{(i,j) \in \mathcal{P}} f(x_i) + f(x_j) \quad (13)$$

where we introduce two sets I and J such that $i \in I$ and $j \in J$ where I and J must follow:

$$I, J \subseteq [\gamma(N), N] \iff I \cup J = [\gamma(N), N] \iff I \cap J = \emptyset$$

The three sets I and J where $i \in I$ and $j \in J$ are each made up of values from the main set, and together they contain all the values from the main set, without sharing any values between them while the third set $\mathcal{P}$ is a subset of the cartesian product of I and J .

Proof. Case 1: Let σ be either injective or bijection on the interval $[\gamma(N), N]$. Since all $f(x_n) \in \mathbb{R}_{\geq 0}$, the original series is non-negative. By applying Definition 3, any rearrangement of a non-negative series preserves the total sum:

$$\sum_{n=\gamma(N)}^N f(x_n) = \sum_{n=\gamma(N)}^N f(x_{\sigma(n)}) = M$$

Our goal for this case is to determine that:

$$\sum_{(i,j) \in \mathcal{P}} f(x_i) + f(x_j) = M$$

Given that σ is at least injective on the interval, the image of function σ is exactly the interval, and since $I \cup J$ equals that interval by construction, we can write where set B is the codomain of our function:

$$\sigma : I \cup J \rightarrow B \quad \text{where} \quad I \cup J \subseteq B \mid \text{Im}(\sigma) = I \cup J, \quad \text{since} \quad \forall n \in I \cup J, \exists m \in I \cup J \mid \sigma(m) = n$$

What can we articulate based on this? We can articulate this based on defining a different expression for n . We want to show and later on prove that this expression holds:

$$\bigcup_{(i,j) \in \mathcal{P}} \{i, j\} = \text{Im}(\sigma) \iff \forall n \in \text{Im}(\sigma), \exists (i, j) \in \mathcal{P} \mid n = i \vee n = j$$

where

$$\sum_{n=\gamma(N)}^N f(x_{\sigma(n)}) = \sum_{n \in \text{Im}(\sigma)} f(x_n) = \sum_{n \in I \cup J} f(x_n) = \sum_{i \in I} f(x_i) + \sum_{j \in J} f(x_j) = M \therefore \sum_{n=\gamma(N)}^N f(x_n) = M = \sum_{i \in I} f(x_i)$$

Now, let's derive further:

$$\sum_{i \in I} f(x_i) + \sum_{j \in J} f(x_j) = \sum_{(i,j) \in \mathcal{P}} f(x_i) + f(x_j) = M$$

Thus, we have now concluded what we needed to do for this case.

Case 2: Now that we solved case 1, let's rewrite our A_N such that

$$A_N = \sum_{n=0}^N f(x_n) = \Gamma(N) + \sum_{n=\gamma(N)}^N f(x_n)$$

where we prove based for each $\Gamma(N)$ on whether $\Gamma(N) = 0$ or $\Gamma(N) = 1$, and show that both cases of $\Gamma(N)$ hold, therefore proving our rewritten A_N holds.

- Let $\Gamma(N) = 0$ where

$$f(x_0) + \sum_{n=1}^N f(x_n) = \sum_{n=0}^N f(x_n) = A_N \quad (14)$$

therefore, A_N holds for this case.

- Let $\Gamma(N) = 1$ where

$$0 + \sum_{n=0}^N f(x_n) = \sum_{n=0}^N f(x_n) = A_N \quad (15)$$

therefore, A_N holds for this case.

Thus, A_N holds for both forms. Therefore if we combine both cases together:

$$A_N = \Gamma(N) + \sum_{(i,j) \in \mathcal{P}} f(x_i) + f(x_j) = \Gamma(N) + \sum_{n=\gamma(N)}^N f(x_n) = \sum_{n=0}^N f(x_n)$$

such that we have proven that the rewritten form of A_N holds. $\square$

Lemma 4. *Later in this sub-section, we want to construct a custom series, such that first fully, we need to define a I and J , which both I and J must follow the rules defined in Lemma 3 to be eligible to be used to construct a custom series. Let's present the given I and J that we are working with:*

$$I = \{2m + \gamma(N) \mid m \in \mathbb{N}_0, 0 \leq 2m \leq N - \gamma(N)\} \quad J = \{2m + 1 - \gamma(N) \mid m \in \mathbb{N}_0, \gamma(N) \leq 2m - \gamma(N) \leq N - \gamma(N)\}$$

Now, let's introduce the rules given for I and J where each item in our list must hold:

1. $I, J \subseteq [\gamma(N), N]$
2. $I \cup J = [\gamma(N), N]$
3. $I \cap J = \emptyset$
4. We are able to introduce a set $\mathcal{P}$ that satisfies Definition 6.

Prove by induction that our now defined sets I and J hold.

Proof. Let $P(n)$, no correlation with set $\mathcal{P}$, be our N where $P(n) = n = N$. At each stage of this proof we will show you that our defined I and J holds.

Base Case 1: When $n = 1$, our $I = \{0\}$ and $J = \{1\}$, such that $[\gamma(N), N] = [0, 1] = \{0, 1\}$. Let's go through the list confirm each one one-by-one:

1. Since $I = \{0\}$ and $J = \{1\}$, both sets are indeed subsets of $[\gamma(N), N]$, thereby satisfying this case since no two elements match.
2. Since $I = \{0\}$ and $J = \{1\}$, $I \cup J = \{0, 1\}$, we have $I \cup J = \{0, 1\}$, and since $\{0, 1\} = [\gamma(N), N]$, this satisfies the union condition.
3. Since $I = \{0\}$ and $J = \{1\}$, since $\{0\} \not\subseteq \{1\}$ and $\{1\} \not\subseteq \{0\}$ then $\{0\} \cap \{1\} = \emptyset$. Therefore, $I \cap J = \{0\} \cap \{1\} = \emptyset$, satisfying this case.
4. Since $I = \{0\}$ and $J = \{1\}$, then $\mathcal{P} \subseteq \{0, 1\}$ because $I \times J = \{0, 1\}$. Now, we need to find some $\mathcal{P}$ such that it satisfies:

$$\bigcup_{(i,j) \in \mathcal{P}} \{i, j\} = I \cup J = \{0, 1\}$$

Let $\mathcal{P} = \{0, 1\}$, therefore:

$$\bigcup_{(i,j) \in \{0,1\}} \{i, j\} = I \cup J = \{0, 1\}$$

thereby satisfying this case.

So $P(1)$ is correct.

Induction hypothesis: Assume for some $k \in \mathbb{N}_0$, the sets

$$I_k = \{2m + \gamma(k) \mid m \in \mathbb{N}_0, 0 \leq 2m \leq k - \gamma(k)\} \quad J_k = \{2m + 1 - \gamma(k) \mid m \in \mathbb{N}_0, \gamma(k) \leq 2m - \gamma(k) \leq k - 1\}$$

satisfy:

1. Subset condition: $I_k, J_k \subseteq [\gamma(k), k]$
2. Union coverage: $I_k \cup J_k = [\gamma(k), k]$
3. Disjointness: $I_k \cap J_k = \emptyset$
4. Set $\mathcal{P}$: there exists a $\mathcal{P}$ that satisfies Definition 6.

Induction step: We will now show that $P(k+1)$ where we defined sets I and J hold. At each stage keep in mind what we are aiming for. So in this case, both sets I_k and J_k are:

$$I_{k+1} = \{2m + \gamma(k+1) \mid m \in \mathbb{N}_0, 0 \leq 2m \leq k+1 - \gamma(k+1)\} \quad J_{k+1} = \{2m + 1 - \gamma(k+1) \mid m \in \mathbb{N}_0, \gamma(k+1) \leq 2m - \gamma(k+1) \leq k\}$$

since we have replaced both n and N with $(k+1)$. We must show that each item on the list holds:

1. Subset condition: $I_{k+1}, J_{k+1} \subseteq [\gamma(k+1), k+1]$
2. Union coverage: $I_{k+1} \cup J_{k+1} = [\gamma(k+1), k+1]$
3. Disjointness: $I_{k+1} \cap J_{k+1} = \emptyset$
4. Set $\mathcal{P}$: there exists a $\mathcal{P}$ that satisfies Definition 6.

To begin, let's acknowledge our bounds:

- $2m \geq 0$ and $2m \leq k + \gamma(k+1)$
- $2m + 1 \leq k + 1 - \gamma(k+1)$ and $2m + 1 \geq 1$

Since $2m$ for both sets is always less or equal to $k+1$, and both I_{k+1}, J_{k+1} contain elements that are greater or equal to $\gamma(N+1)$, the inclusion holds for $I_{k+1}, J_{k+1} \subseteq [\gamma(k+1), k+1]$. So by working through each item on our list, we must show that each one holds:

- Union coverage:

- Case 1: $k+1$ is odd ($\gamma(k+1) = 0$): We can define our sets I_{k+1} and J_{k+1} such that

$$I_{k+1} = \{2m + 0 \mid m \in \mathbb{N}_0, 0 \leq 2m \leq k+1-0\} \quad J_{k+1} = \{2m + 1 - 0 \mid m \in \mathbb{N}_0, 0 \leq 2m - 0 \leq k\}$$

where $I_{k+1} = \{0, 2, \dots, k\}$, where k is the largest even integer less than $k+1$, and $J_{k+1} = \{1, 3, \dots, k+1\}$. Therefore, when we take the union of both sets where $I_{k+1} \cup J_{k+1} = \{0, 1, 2, 3, \dots, k, k+1\}$. Since our $[\gamma(k+1), k+1] = [0, k+1] = \{0, 1, 2, 3, \dots, k, k+1\}$, then

$$[\gamma(k+1), k+1] = I_{k+1} \cup J_{k+1}$$

Thus, the union coverage holds for this case.

- Case 2: $k+1$ is even ($\gamma(k+1) = 1$): We can define our sets I_{k+1} and J_{k+1} such that

$$I = \{2m \mid m \in \mathbb{N}_0, 0 \leq 2m \leq k\} \quad \text{and} \quad J = \{2m \mid m \in \mathbb{N}_0, 1 \leq 2m - 1 \leq k\}$$

where $I_{k+1} = \{1, 3, \dots, k\}$, where k is the largest odd integer less than $k+1$, and $J_{k+1} = \{2, 4, \dots, k+1\}$. Therefore, when we take the union of both sets where $I_{k+1} \cup J_{k+1} = \{1, 2, 3, 4, \dots, k, k+1\}$. Since our $[\gamma(k+1), k+1] = [1, k+1] = \{1, 2, 3, 4, \dots, k, k+1\}$, then

$$[\gamma(k+1), k+1] = I_{k+1} \cup J_{k+1}$$

Thus, the union coverage holds for this case.

- Disjointness: As shown in our previous case, since I_{k+1} consists entirely of even natural numbers and J_{k+1} consists entirely of odd natural numbers, their intersection results in an empty set:

$$I_{k+1} \cap J_{k+1} = \emptyset$$

Thus, the disjointness condition holds.

- Set $\mathcal{P}$:

- Case 1: $k + 1$ is odd ($\gamma(k + 1) = 0$): We can define our sets I_{k+1} and J_{k+1} such that

$$I_{k+1} = \{2m \mid m \in \mathbb{N}_0, 0 \leq 2m \leq k\} \quad J_{k+1} = \{2m + 1 \mid m \in \mathbb{N}_0, 1 \leq 2m + 1 \leq k + 1\}$$

where $I_{k+1} = \{0, 2, \dots, k\}$, where k is the largest even integer less than $k + 1$, and $J_{k+1} = \{1, 3, \dots, k + 1\}$. Since $\mathcal{P}_{k+1} \subseteq I_{k+1} \times J_{k+1}$, there exists a set $\mathcal{P}_{k+1} = \{(0, 1), (2, 3), \dots, (k, k + 1)\}$ such that no two ordered pairs in $\mathcal{P}_{k+1}$ share the same first or second coordinate as shown in Definition 6. Thus:

$$\bigcup_{(i,j) \in \mathcal{P}_{k+1}} \{i, j\} = \{0, 1\} \cup \{2, 3\} \cup \dots \cup \{k, k + 1\} = \{0, 1, 2, 3, \dots, k, k + 1\} = I_{k+1} \cup J_{k+1}$$

where there exists some $\mathcal{P}_{k+1}$ that holds.

- Case 2: $k + 1$ is even ($\gamma(k + 1) = 1$): We can define our sets I_{k+1} and J_{k+1} such that

$$I_{k+1} = \{2m \mid m \in \mathbb{N}_0, 2 \leq 2m \leq k + 1\} \quad J_{k+1} = \{2m + 1 \mid m \in \mathbb{N}_0, 1 \leq 2m + 1 \leq k\}$$

where $I_{k+1} = \{2, 4, \dots, k + 1\}$, where k is the largest odd integer less than $k + 1$, and $J_{k+1} = \{1, 3, \dots, k\}$. Since $\mathcal{P}_{k+1} \subseteq I_{k+1} \times J_{k+1}$, there exists a set $\mathcal{P}_{k+1} = \{(1, 2), (3, 4), \dots, (k, k + 1)\}$ such that no two ordered pairs in $\mathcal{P}_{k+1}$ share the same first or second coordinate as shown in Definition 6. Thus:

$$\bigcup_{(i,j) \in \mathcal{P}_{k+1}} \{i, j\} = \{1, 2\} \cup \{3, 4\} \cup \dots \cup \{k, k + 1\} = \{1, 2, 3, 4, \dots, k, k + 1\} = I_{k+1} \cup J_{k+1}$$

where there exists some $\mathcal{P}_{k+1}$ that holds.

So $P(k + 1)$ is correct. Hence by mathematical induction $P(n)$ is correct for all non-negative integer n . $\square$

Lemma 5. *Later in this sub-section, we want to construct a custom series, such that first fully, we need to define a I and J , which both I and J must follow the rules defined in Lemma 3 to be eligible to be used to construct a custom series. Let's present the given I and J that we are working with:*

$$I = \left\{ m \mid m \in \mathbb{N}_0, \gamma(N) \leq m \leq \left\lfloor \frac{N}{2} \right\rfloor \right\} \quad \text{and} \quad J = \left\{ N - m \mid m \in \mathbb{N}_0, 0 \leq m \leq \left\lfloor \frac{N}{2} \right\rfloor - \gamma(N) \right\}$$

Now, let's introduce the rules given for I and J where each item in our list must hold:

1. $I, J \subseteq [\gamma(N), N]$
2. $I \cup J = [\gamma(N), N]$
3. $I \cap J = \emptyset$
4. We are able to introduce a set $\mathcal{P}$ that satisfies Definition 6.

Prove by induction that our now defined sets I and J hold.

Proof. Let $P(n)$ be our N where $P(n) = n = N$. At each stage of this proof we will show you that our defined I and J holds.

Base Case 1: When $n = 1$, our $I = \{0\}$ and $J = \{1\}$, such that $[\gamma(N), N] = [0, 1] = \{0, 1\}$. Let's go through the list confirm each one one-by-one:

1. Since $I = \{0\}$ and $J = \{1\}$, both sets are indeed subsets of $[\gamma(N), N]$, thereby satisfying this case since no two elements match.

2. Since $I = \{0\}$ and $J = \{1\}$, $I \cup J = \{0, 1\}$, we have $I \cup J = \{0, 1\}$, and since $\{0, 1\} = [\gamma(N), N]$, this satisfies the union condition.
3. Since $I = \{0\}$ and $J = \{1\}$, since $\{0\} \not\subseteq \{1\}$ and $\{1\} \not\subseteq \{0\}$ then $\{0\} \cap \{1\} = \emptyset$. Therefore, $I \cap J = \{0\} \cap \{1\} = \emptyset$, satisfying this case.
4. Since $I = \{0\}$ and $J = \{1\}$, then $\mathcal{P} \subseteq \{0, 1\}$ because $I \times J = \{0, 1\}$. Now, we need to find some $\mathcal{P}$ such that it satisfies:

$$\bigcup_{(i,j) \in \mathcal{P}} \{i, j\} = I \cup J = \{0, 1\}$$

Let $\mathcal{P} = \{0, 1\}$, therefore:

$$\bigcup_{(i,j) \in \{0,1\}} \{i, j\} = I \cup J = \{0, 1\}$$

thereby satisfying this case.

So $P(1)$ is correct.

Induction hypothesis: Assume for some $k \in \mathbb{N}_0$, the sets

$$I_k = \left\{ m \mid m \in \mathbb{N}_0, \gamma(k) \leq m \leq \left\lfloor \frac{k}{2} \right\rfloor \right\} \quad \text{and} \quad J_k = \left\{ k - m \mid m \in \mathbb{N}_0, 0 \leq m \leq \left\lfloor \frac{k}{2} \right\rfloor - \gamma(k) \right\}$$

satisfy:

1. Subset condition: $I_k, J_k \subseteq [\gamma(k), k]$
2. Union coverage: $I_k \cup J_k = [\gamma(k), k]$
3. Disjointness: $I_k \cap J_k = \emptyset$
4. Set $\mathcal{P}$: there exists a $\mathcal{P}$ that satisfies Definition 6.

Induction step: We will now show that $P(k+1)$ where we defined sets I and J hold. At each stage keep in mind what we are aiming for. So in this case, both sets I_k and J_k are:

$$I_{k+1} = \left\{ m \mid m \in \mathbb{N}_0, \gamma(k+1) \leq m \leq \left\lfloor \frac{k+1}{2} \right\rfloor \right\} \quad \text{and} \quad J_{k+1} = \left\{ k+1 - m \mid m \in \mathbb{N}_0, 0 \leq m \leq \left\lfloor \frac{k+1}{2} \right\rfloor - \gamma(k+1) \right\}$$

since we have replaced both n and N with $(k+1)$. We must show that each item on the list holds:

1. Subset condition: $I_{k+1}, J_{k+1} \subseteq [\gamma(k+1), k+1]$
2. Union coverage: $I_{k+1} \cup J_{k+1} = [\gamma(k+1), k+1]$
3. Disjointness: $I_{k+1} \cap J_{k+1} = \emptyset$
4. Set $\mathcal{P}$: there exists a $\mathcal{P}$ that satisfies Definition 6.

To begin, let's acknowledge our bounds:

- $m \geq \gamma(k+1)$ and $m \leq \left\lfloor \frac{k+1}{2} \right\rfloor$
- $m \geq \left\lfloor \frac{k+1}{2} \right\rfloor + 1$ and $m \leq k+1$

Since $2m$ for both sets is always less or equal to $k + 1$, and both I_{k+1}, J_{k+1} contain elements that are greater or equal to $\gamma(N + 1)$, the inclusion holds for $I_{k+1}, J_{k+1} \subseteq [\gamma(k + 1), k + 1]$. So by working through each item on our list, we must show that each one holds:

- Union coverage:

- Case 1: $k + 1$ is odd ($\gamma(k + 1) = 0$): We can define our sets I_{k+1} and J_{k+1} such that

$$\left\{ m \mid m \in \mathbb{N}_0, 0 \leq m \leq \left\lfloor \frac{k+1}{2} \right\rfloor \right\} \quad \text{and} \quad \left\{ k+1-m \mid m \in \mathbb{N}_0, 0 \leq m \leq \left\lfloor \frac{k+1}{2} \right\rfloor - 0 \right\}$$

where $I_{k+1} = \left\{ 0, 1, \dots, \left\lfloor \frac{k+1}{2} \right\rfloor \right\}$ and $J_{k+1} = \left\{ k+1, k, \dots, \left\lfloor \frac{k+1}{2} \right\rfloor + 1 \right\}$ in which:

$$k+1 - \left\lfloor \frac{k+1}{2} \right\rfloor = \left\lfloor \frac{k+1}{2} \right\rfloor + 1k - \left\lfloor \frac{k+1}{2} \right\rfloor = \left\lfloor \frac{k+1}{2} \right\rfloor k = 2 \left\lfloor \frac{k+1}{2} \right\rfloor \quad \because k+1 \not\equiv 2 \iff \left\lfloor \frac{k+1}{2} \right\rfloor$$

Therefore, when we take the union of both sets where $I_{k+1} \cup J_{k+1} = \left\{ 0, 1, \dots, \left\lfloor \frac{k+1}{2} \right\rfloor, \left\lfloor \frac{k+1}{2} \right\rfloor + 1, \dots, k, k+1 \right\}$ where $0 \leq \left\lfloor \frac{k+1}{2} \right\rfloor \leq k+1$. Since our $[\gamma(k+1), k+1] = [0, k+1] = \left\{ 0, 1, \dots, \left\lfloor \frac{k+1}{2} \right\rfloor, \left\lfloor \frac{k+1}{2} \right\rfloor + 1, \dots, k, k+1 \right\}$, then

$$[\gamma(k+1), k+1] = I_{k+1} \cup J_{k+1}$$

Thus, the union coverage holds for this case.

- Case 2: $k + 1$ is even ($\gamma(k + 1) = 1$): We can define our sets I_{k+1} and J_{k+1} such that

$$I_{k+1} = \left\{ m \mid m \in \mathbb{N}_0, 1 \leq m \leq \left\lfloor \frac{k+1}{2} \right\rfloor \right\} \quad \text{and} \quad J_{k+1} = \left\{ k+1-m \mid m \in \mathbb{N}_0, 0 \leq m \leq \left\lfloor \frac{k+1}{2} \right\rfloor - 1 \right\}$$

where $I = \left\{ 1, 2, \dots, \left\lfloor \frac{k+1}{2} \right\rfloor \right\}$ and $J_{k+1} = \left\{ k+1, k, \dots, \left\lfloor \frac{k+1}{2} \right\rfloor + 1 \right\}$. Therefore, when we take the union of both sets where $I_{k+1} \cup J_{k+1} = \left\{ 1, 2, \dots, \left\lfloor \frac{k+1}{2} \right\rfloor, \left\lfloor \frac{k+1}{2} \right\rfloor + 1, \dots, k, k+1 \right\}$ where $0 \leq \left\lfloor \frac{k+1}{2} \right\rfloor \leq k+1$. Since our $[\gamma(k+1), k+1] = [1, k+1] = \left\{ 1, 2, \dots, \left\lfloor \frac{k+1}{2} \right\rfloor, \left\lfloor \frac{k+1}{2} \right\rfloor + 1, \dots, k, k+1 \right\}$, then

$$[\gamma(k+1), k+1] = I_{k+1} \cup J_{k+1}$$

Thus, the union coverage holds for this case.

- Disjointness: As shown in the previous case, the largest element of I_{k+1} is $\left\lfloor \frac{k+1}{2} \right\rfloor$, and the smallest element of J_{k+1} is $\left\lfloor \frac{k+1}{2} \right\rfloor + 1$, for all values of $\gamma(k + 1)$. Therefore, since $\left\lfloor \frac{k+1}{2} \right\rfloor < \left\lfloor \frac{k+1}{2} \right\rfloor + 1$ then no two sets share the same value to where

$$I_{k+1} \cap J_{k+1} = \emptyset$$

Thus, the disjointness condition holds.

- Set $\mathcal{P}$:

- Case 1: $k+1$ is odd ($\gamma(k+1) = 0$): We can define our sets I_{k+1} and J_{k+1} such that

$$I_{k+1} = \left\{ m \mid m \in \mathbb{N}_0, 0 \leq m \leq \left\lfloor \frac{k+1}{2} \right\rfloor \right\} \quad \text{and} \quad J_{k+1} = \left\{ k+1-m \mid m \in \mathbb{N}_0, 0 \leq m \leq \left\lfloor \frac{k+1}{2} \right\rfloor - 0 \right\}$$

where $I_{k+1} = \left\{ 0, 1, \dots, \left\lfloor \frac{k+1}{2} \right\rfloor \right\}$ and $J_{k+1} = \left\{ k+1, k, \dots, \left\lfloor \frac{k+1}{2} \right\rfloor + 1 \right\}$. Since $\mathcal{P}_k \subseteq I_{k+1} \times J_{k+1}$, there exists a set $\mathcal{P}_{k+1} = \left\{ (0, k+1), (1, k), \dots, \left(\left\lfloor \frac{k+1}{2} \right\rfloor, \left\lfloor \frac{k+1}{2} \right\rfloor + 1 \right) \right\}$ such that no two ordered pairs in $\mathcal{P}_{k+1}$ share the same first or second coordinate as shown in Definition 6. Thus:

$$\bigcup_{(i,j) \in \mathcal{P}_{k+1}} \{i, j\} = \{0, k+1\} \cup \{1, k\} \cup \dots \cup \left\{ \left\lfloor \frac{k+1}{2} \right\rfloor, \left\lfloor \frac{k+1}{2} \right\rfloor + 1 \right\} = I_{k+1} \cup J_{k+1}$$

where there exists some $\mathcal{P}_{k+1}$ that holds.

- Case 2: $k+1$ is even ($\gamma(k+1) = 1$): We can define our sets I_{k+1} and J_{k+1} such that

$$I_{k+1} = \left\{ m \mid m \in \mathbb{N}_0, 1 \leq m \leq \left\lfloor \frac{k+1}{2} \right\rfloor \right\} \quad \text{and} \quad J_{k+1} = \left\{ k+1-m \mid m \in \mathbb{N}_0, 0 \leq m \leq \left\lfloor \frac{k+1}{2} \right\rfloor - 1 \right\}$$

where $I_{k+1} = \left\{ 1, 2, \dots, \left\lfloor \frac{k+1}{2} \right\rfloor \right\}$ and $J_{k+1} = \left\{ k+1, k, \dots, \left\lfloor \frac{k+1}{2} \right\rfloor + 1 \right\}$. Since $\mathcal{P}_k \subseteq I_{k+1} \times J_{k+1}$, there exists a set $\mathcal{P}_{k+1} = \left\{ (1, k+1), (2, k), \dots, \left(\left\lfloor \frac{k+1}{2} \right\rfloor, \left\lfloor \frac{k+1}{2} \right\rfloor + 1 \right) \right\}$ such that no two ordered pairs in $\mathcal{P}_{k+1}$ share the same first or second coordinate as shown in Definition 6. Thus:

$$\bigcup_{(i,j) \in \mathcal{P}_{k+1}} \{i, j\} = \{0, k+1\} \cup \{1, k\} \cup \dots \cup \left\{ \left\lfloor \frac{k+1}{2} \right\rfloor, \left\lfloor \frac{k+1}{2} \right\rfloor + 1 \right\} = I_{k+1} \cup J_{k+1}$$

where there exists some $\mathcal{P}_{k+1}$ that holds.

So $P(k+1)$ is correct, Hence by mathematical induction $P(n)$ is correct for all non-negative integer n . $\square$

4.1.1 Validating Even Decomposition

Proposition 2. Let I and J be defined sets proposed in Lemma 4 such that

$$I = \{2m + \gamma(N) \mid m \in \mathbb{N}_0, 0 \leq 2m \leq N - \gamma(N)\} \quad J = \{2m + 1 - \gamma(N) \mid m \in \mathbb{N}_0, \gamma(N) \leq 2m - \gamma(N) \leq N\} \quad (16)$$

Prove that $|I| = |J|$ for all $N \in \mathbb{N}_0$.

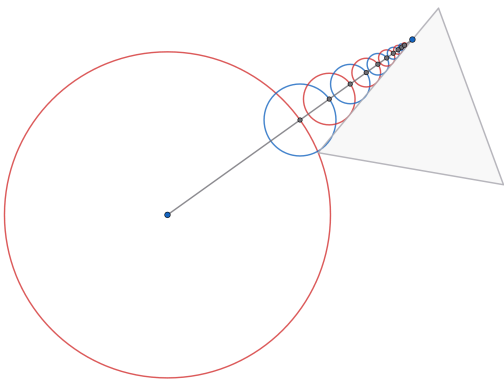
Proof. Suppose that $|I| \neq |J|$ for all $N \in \mathbb{N}_0$. Then, I and J do not contain the same amount of elements despite the value of N in correlation with $\gamma(N)$. To begin, let's introduce two cases based on the two outputs of $\gamma(N)$.

Case 1: Suppose that N is even where $\gamma(N) = 1$. From what we established in Lemma 4, $I = \{1, 3, \dots, N-1\}$ and $J = \{2, 4, \dots, N\}$. If we shift all elements in set I by 1, then $I+1 = \{2, 4, \dots, N\}$. Therefore, $J = I+1$ such that $|J| = |I|$ for this case. Thus, we have reached a contradiction for this case.

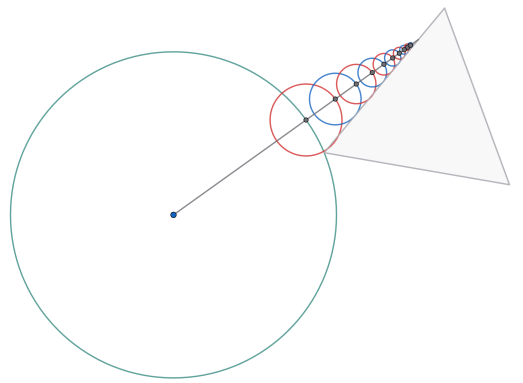
Case 2: Suppose that N is odd where $\gamma(N) = 0$. From what we established in Lemma 4, $I = \{0, 2, \dots, N-1\}$ and $J = \{1, 3, \dots, N\}$. If we shift all the elements in set I by 1, then $I + 1 = \{1, 3, \dots, N\}$. Therefore, $J = I + 1$ such that $|J| = |I|$ for this case. Thus, we have reached a contradiction for this case.

From the cases gathered. Both of them contradict what was stated from our proof where $|I| \neq |J|$ for all $N \in \mathbb{N}_0$. Thus, $|I| = |J|$ for all $N \in \mathbb{N}_0$ instead. $\square$

By what was explained in our Proposition 2, since both I and J have the same cardinality, if they are defined from Lemma 4, this means that $|I \cup J| = 2|I| = 2|J|$ such that $|I \cup J| \equiv 2$. In Figure 3, we can see that if we don't take into account $\Gamma(N)$, then our total iterations N upon reaching ε or $D_{\max}$ will always end up being an even value.



(a) In this figure, our N is an odd number. Therefore, $\Gamma(N) = 0$ where the first traversal is $f(t_0)$ where some $i \in I$ is 0. Later, proceeding with $f(t_1)$ where some $j \in J$ is 1.



(b) In this figure, our N is an even number. Therefore, $\Gamma(N) = f(t_0)$ where the first traversal is $f(t_1)$ where some $i \in I$ is 1. Later, proceeding with $f(t_2)$ where some $j \in J$ is 2.

Figure 3: For each sub-figure, we define the sets I and J according to Lemma 4. Each t_i with $i \in I$ is shown with a red circular outline, each t_j with $j \in J$ is shown with a blue circular outline, and the initial $\Gamma(N)$ s shown in a green circular outline.

Algorithm 2 Ray-marching presented in Figure 3 (Modified algorithm of [7])

```

1:  $t \leftarrow \Gamma(N)$ 
2:  $p \leftarrow r(t) = r_0 + t \cdot r_d$ 
3: if  $t < \varepsilon$  and  $\Gamma(N) \neq 0$  then                                ▷ If  $\Gamma(N) \neq 0$ , then it might hit before reaching our series
4:   return hit at  $p$ 
5: end if
6: for  $n = 0$  to  $\left\lfloor \frac{N}{2} \right\rfloor - 1$  do
7:    $d_i \leftarrow f(p, S)$                                               ▷ For simplicity sake, let  $i = i_n$ 
8:   if  $d_i < \varepsilon$  then
9:     return hit at  $p$ 
10:  end if
11:   $t \leftarrow t + d_i$ 
12:  if  $t > D_{\max}$  then
13:    return no hit (ray escaped)
14:  end if
15:   $p \leftarrow r(t)$ 
16:
17:   $d_j \leftarrow f(p, S)$                                               ▷ For simplicity sake, let  $j = j_n$ 
18:  if  $d_j < \varepsilon$  then
19:    return hit at  $p$ 
20:  end if
21:   $t \leftarrow t + d_j$ 
22:  if  $t > D_{\max}$  then
23:    return no hit (ray escaped)
24:  end if
25:   $p \leftarrow r(t)$ 
26: end for
27: return no hit (ray escaped)

```

Proposition 3. Let I and J be defined sets proposed in Lemma 5 such that

$$I = \left\{ m \mid m \in \mathbb{N}_0, \gamma(N) \leq m \leq \left\lfloor \frac{N}{2} \right\rfloor \right\} \quad \text{and} \quad J = \left\{ N - m \mid m \in \mathbb{N}_0, 0 \leq m \leq \left\lfloor \frac{N}{2} \right\rfloor - \gamma(N) \right\} \quad (17)$$

Prove that $|I| = |J|$ for all $N \in \mathbb{N}_0$.

Proof. Suppose that $|I| \neq |J|$ for all $N \in \mathbb{N}_0$. Then, I and J do not contain the same amount of elements despite the value of N in correlation with $\gamma(N)$. To begin, let's introduce two cases based on the two outputs of $\gamma(N)$.

Case 1: Suppose that N is even where $\gamma(N) = 1$. From what we established in Lemma 5, $I = \left\{ 1, 2, \dots, \left\lfloor \frac{N}{2} \right\rfloor \right\}$ and $J = \left\{ N, N - 1, \dots, \left\lfloor \frac{N}{2} \right\rfloor + 1 \right\}$. If we take $N + 1$ and subtract each element in set I , then $N + 1 - I = \left\{ N + 1 - 1, N + 1 - 2, \dots, N + 1 - \left\lfloor \frac{N}{2} \right\rfloor \right\}$ in which:

$$\begin{aligned}
N + 1 - \left\lfloor \frac{N}{2} \right\rfloor &= \left\lfloor \frac{N}{2} \right\rfloor + 1 \\
N - \frac{N}{2} &= \frac{N}{2} \quad \because N \equiv 2 \iff \left\lfloor \frac{N}{2} \right\rfloor = \frac{N}{2} \\
N &= N
\end{aligned}$$

such that now $N + 1 - I = \left\{ N, N - 1, \dots, \left\lfloor \frac{N}{2} \right\rfloor + 1 \right\}$. Therefore, $J = N + 1 - I$ such that $|J| = |I|$ for this case. Thus, we have reached a contradiction for this case.

Case 2: Suppose that N is odd where $\gamma(N) = 0$. From what we established in Lemma 5, $I = \left\{ 0, 1, \dots, \left\lfloor \frac{N}{2} \right\rfloor \right\}$ and $J = \left\{ N, N - 1, \dots, \left\lfloor \frac{N}{2} \right\rfloor + 1 \right\}$. If we take N and subtract each element in set I , then $N - I = \left\{ N, N - 1, \dots, N - \left\lfloor \frac{N}{2} \right\rfloor \right\}$ in which:

$$\begin{aligned} N - \left\lfloor \frac{N}{2} \right\rfloor &= \left\lfloor \frac{N}{2} \right\rfloor + 1 \\ N - \frac{N-1}{2} &= \frac{N-1}{2} + 1 \quad \because N \not\equiv 2 \iff \left\lfloor \frac{N}{2} \right\rfloor = \frac{N-1}{2} \\ N &= N - 1 + 1 \\ N &= N \end{aligned}$$

Therefore, $J = N - I$ such that $|J| = |I|$ for this case. Thus, we have reached a contradiction for this case.

From the cases gathered. Both of them contradict what was stated from our proof where $|I| \neq |J|$ for all $N \in \mathbb{N}_0$. Thus, $|I| = |J|$ for all $N \in \mathbb{N}_0$ instead. $\square$

4.2 Selection Algorithm for Ray-marching

In this sub-section, we will introduce the concept of bi-directional ray intersection. Therefore, we will redefine N to represent the number of steps taken until our ray converges or intersects. For better understanding, we will define two lists that are variations for set J where $J^\rightarrow$ and $J^\leftarrow$.

Now, let's construct the series for t_T that stems from t_f but instead of using tropical semirings from Definition 2, we instead construct it from Lemma 3:

$$t_T = \int_S g(r(t)) dt \approx \sum_{n=0}^N f(t_n, S) \quad \text{where } f(t, S) = g(r(t)) \cdot \text{SDF}(r(t), S) \approx \Gamma(N) + \sum_{(i,j) \in \mathcal{P}} f(t_i, S) + f(t_j, S) \quad (18)$$

where $t_T \stackrel{?}{=} t_f$. The purpose of t_T is to represent the result if we were to accumulate every value in sets I and J in both directions; this implies that t_T is a theoretical value and only used for conceptual purposes.

Definition 7. Let $J^\rightarrow$ and $J^\leftarrow$ be ordered lists (from least to greatest) whose ranges are disjoint subsets of the set J ; that is,

$$\text{range}(J^\rightarrow) \subseteq J, \quad \text{range}(J^\leftarrow) \subseteq J, \quad \text{range}(J^\rightarrow) \cap \text{range}(J^\leftarrow) = \emptyset, \quad \text{and} \quad \text{range}(J^\rightarrow) \cup \text{range}(J^\leftarrow) = J.$$

While the series associated with $J^\rightarrow$ begins at t_0 and progresses toward $D_{\max}$, the sequence $J^\leftarrow$ instead begins at $D_{\max}$ and moves in reverse toward t_0 . For the sake of analysis:

- $J^\rightarrow$ refers to the portion of J used in the summation from Lemma 4
- $J^\leftarrow$ refers to the portion of J used in the convergence behavior established by Lemma 5

Definition 8. Let's define a sorted list for I called L_I , which means that $I = \text{range}(L_I)$ where $L_I = (i_0, i_1, \dots, i_n)$. Our total amount of indexes of forward ray-marching traversal such that $L_I \mathbin{++} J^\rightarrow = (k_0, k_1, \dots, k_n)$. Using tropical semirings defined in Definition 2, let $\Delta t_{k_{n+1}}^\oplus = \Delta t_{k_n}^\oplus \oplus \Delta t_{k_{n+1}}$ where $\Delta t_{k_0}^\oplus = \Delta t_0^\oplus = D_{\max}$. We define $\Delta t^\oplus$ to find the minimum distance per iteration to detect change in growth.

To define a preliminary concept, let L_T be an ordered list for all our iterations that progressed us to t_T where $(0, 1, \dots, T)$ and define it as $L_T = L_I \mathbin{++} J^\rightarrow \mathbin{++} J^\leftarrow$ such that $L_T = \{i_0, j_0, i_1, j_1, \dots, i_n, j_n\}$. In our case, we want to define a logical way to store our j index to either list $J^\rightarrow$ or $J^\leftarrow$. Our goal is to predict a small increase in Δt that might signify that our ray was heading closer to a surface but later traversing farther from the object, making it potentially more likely for our ray to diverge. Our small change in Δt can be expressed, using Definition 8, as:

$$\delta(d_i, \Delta t^\oplus) = d_i - \Delta t^\oplus \quad (19)$$

For every instance $\delta(d_i, \Delta t^\oplus) > \delta_{\min}$, our follow up iteration will be pushed to the list $J^\leftarrow$, else, if k_0 or $\delta(\Delta t_{k_n}) < \delta_{\min}$, our follow up iteration will be pushed to the list $J^\rightarrow$.

Eventually, the quantities Δt_k and $\Delta t_{j^\leftarrow}$ where $j^\rightarrow \in J^\rightarrow$ and $j^\leftarrow \in J^\leftarrow$, will intersect via distance field evaluation, as shown in Figure 4. Alternatively, convergence to a surface may occur from either direction independently; in this case, it is given that both distance fields from opposing sides will never intersect with each other. In Algorithm 3, we summarize everything in this sub-section and tie everything in this section together.

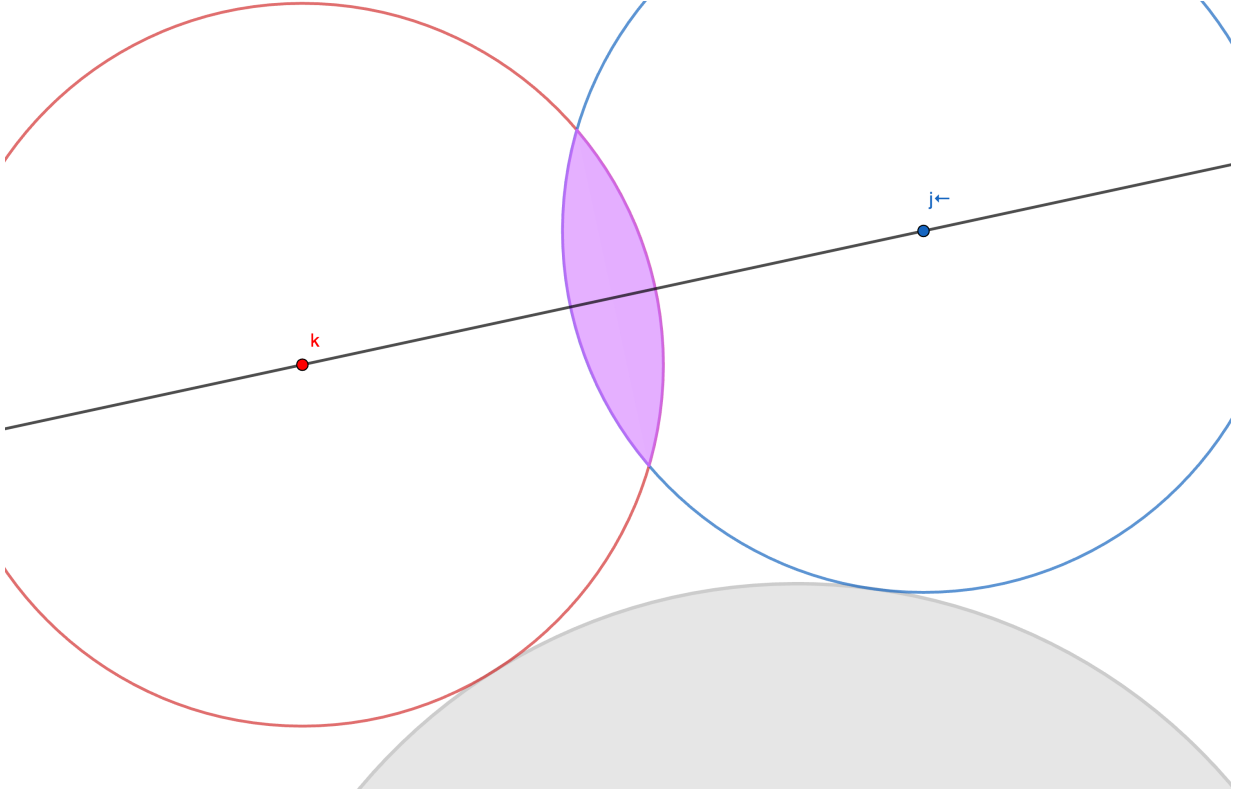


Figure 4: The figure illustrates the visual intersection between Δt_k and $\Delta t_{j^\leftarrow}$, where Δt_k denotes the radius of the distance field to circle k and $\Delta t_{j^\rightarrow}$ denotes the radius of our distance field to circle $j^\rightarrow$. The purple intersection indicates that both circles have overlapped, signaling that continued progression in this direction would ultimately lead to divergence.

Since when two opposing distance fields intersect with each other, we know ahead of time if our ray is going to diverge, as illustrated in Figure 4. Therefore, we can exit our series. The reason

this is important is that we can likely reduce the number of iterations used for the ray that will diverge, and a diverging ray results in wasted computation. That is what we are trying to achieve with Algorithm 3.

Algorithm 3 Two-Way Ray-marching (Modified algorithm of [7])

```

1:  $t^{\rightarrow} \leftarrow \Gamma(N)$ 
2:  $p^{\rightarrow} \leftarrow r(t^{\rightarrow}) = r_0 + t^{\rightarrow} \cdot r_d$ 
3: if  $t^{\leftarrow} < \varepsilon$  and  $\Gamma(N) \neq 0$  then           ▷ If  $\Gamma(N) \neq 0$ , then it might hit before reaching our series
4:   return hit at  $p$ 
5: end if
6:  $t^{\leftarrow} \leftarrow D_{\max}$ 
7:  $\Delta t^{\oplus} \leftarrow D_{\max}$ 
8:  $n \leftarrow \gamma(N)$ 
9: while  $n \leq N$  do
10:   $k \leftarrow n$                                 ▷ In this line,  $k \in \text{range}(L_I)$ 
11:   $d_k \leftarrow f(p^{\rightarrow}, S)$ 
12:  if  $d_k < \varepsilon$  then
13:    return hit at  $p$ 
14:  end if
15:   $t^{\rightarrow} \leftarrow t^{\rightarrow} + d_k$ 
16:  if  $t^{\rightarrow} > D_{\max}$  then
17:    return hit at  $p$ 
18:  end if
19:   $\Delta t^{\oplus} \leftarrow \Delta t^{\oplus} \oplus d_i$ 
20:   $p^{\rightarrow} \leftarrow r(t^{\rightarrow})$ 
21:   $n \leftarrow n + 1$ 
22:  if  $d_i - \Delta t^{\oplus} < \delta_{\min}$  then           ▷ This is our  $\delta$  growth check
23:     $p^{\leftarrow} \leftarrow r(t^{\leftarrow}) = r_0 + t^{\leftarrow} \cdot r_d$ 
24:     $j^{\leftarrow} \leftarrow n$ 
25:     $d_{j^{\leftarrow}} \leftarrow f(p^{\leftarrow}, S)$ 
26:    if  $d_i + d_{j^{\leftarrow}} \geq |t^{\leftarrow} - t^{\rightarrow}|$  then           ▷ This line checks if the distance fields intersected
27:      return no hit (ray escaped)
28:    end if
29:     $t^{\leftarrow} \leftarrow t^{\leftarrow} - d_{j^{\leftarrow}}$ 
30:  else
31:     $k \leftarrow n$                                 ▷ In this line,  $k \in \text{range}(J^{\rightarrow})$ 
32:     $d_k \leftarrow f(p^{\rightarrow}, S)$ 
33:    if  $d_k < \varepsilon$  then
34:      return hit at  $p$ 
35:    end if
36:     $t^{\rightarrow} \leftarrow t^{\rightarrow} + d_k$ 
37:    if  $t^{\rightarrow} > D_{\max}$  then
38:      return hit at  $p$ 
39:    end if
40:     $\Delta t^{\oplus} \leftarrow \Delta t^{\oplus} \oplus d_i$ 
41:     $p^{\rightarrow} \leftarrow r(t^{\rightarrow})$ 
42:  end if
43:   $n \leftarrow n + 1$ 
44: end while
45: return no hit (ray escaped)

```

5 System and Application

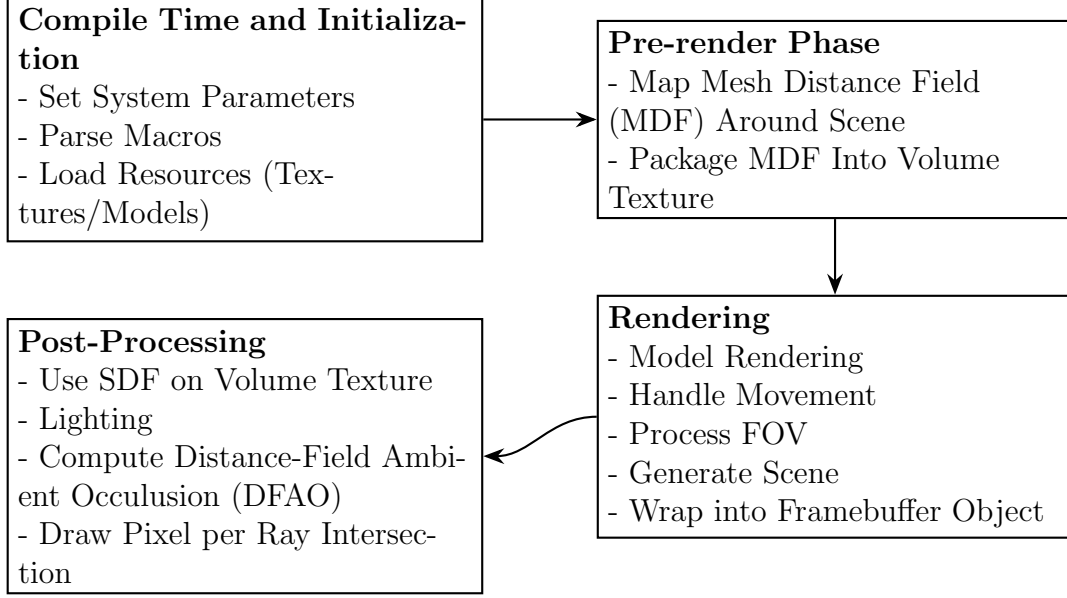


Figure 5: The system flow state for the Two-Way Ray-marching application pipeline for processing Distance-Field Ambient Occlusion.

5.1 System and Application Design

The Two-Way Ray-marching application was constructed using C99, with Premake5. The graphics pipeline is configured to utilize either OpenGL 4.1 or OpenGL ES 3.0, with WebGL 2.0 support enabled via Emscripten. Shaders are written in GLSL. The research was conducted on a macOS Sequoia platform with a 64-bit architecture, utilizing the Apple M1 Pro Chip.

The application builds and renders an interactive scene through a four-stage process, as illustrated in the diagram shown in Figure 5. The compile-time configuration is fully modifiable through `config.h` in the `include` directory, see [5]. The configuration file—a header file written in C—provides a range of macros that allow the user to control how the scene is processed and drawn, including the grid size that makes our Mesh Distance Field and the location of our vertex attribute location constants. The `FRAGMENT_SELECTOR` macro can be used to determine which level of our scene should be processed, whether it is a primitive or a mesh with lighting computed and rendered using distance-field ray-marching.

Based on the Compile-Time and initialization phases of the algorithm, the application generates object models and distributes them through the Pre-processing phase. Next, based on the Pre-processing phase, the model will be mapped using a Mesh Distance Field (MDF) through OpenCL and later stored in a large 32-bit float buffer, packaged into a 3D texture, which we refer to as the Volume Texture [4]. In Unreal Engine, the volume texture is a 2D texture array.

To initiate ray-marching through the volume texture, we first construct a ray direction for each pixel by transforming screen-space coordinates into world space. To construct this, we transform normalized device coordinates into world space using the inverse of the projection matrix C_P and view matrix C_V . Let $x_{clip} = [x, y, -1, 1]^T$ represent the pixel position in clip space. The world-space ray direction r_d is then computed as:

$$r_d = \text{normalize}\left((C_V^{-1}C_P^{-1}x_{clip})_{xyz}\right) \quad (20)$$

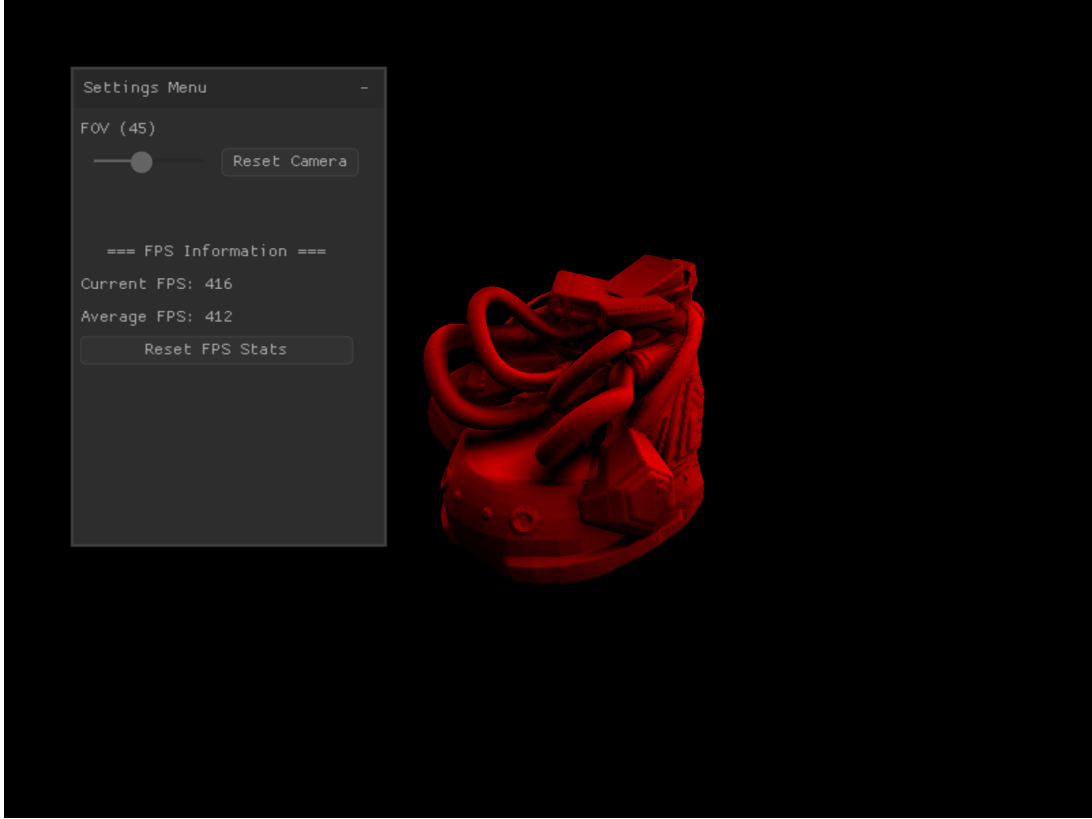


Figure 6: View of the running application using Distance-Field Ambient Occlusion alongside basic lighting on a mesh. The model is named the Damaged Helmet.

6 Results and Discussions

A series of tests were conducted to assess the performance efficiency of our ray marching algorithm in various types of hardware. The evaluation focused on two key parameters: the minimum growth factor applied during the rendering process and the camera’s field of view (FOV). To measure the algorithm’s effectiveness, we rendered two distinct models: a torus and a damaged helmet mesh. These tests provide insights into the algorithm’s computational performance under varying conditions.

6.1 Efficiency

When it comes to hardware testing, we compare the number of frames generated by our two-way ray-marching algorithm with that of standard ray-marching. We evaluate the average frames per second generated based on a sample pool of the last 480 frames with recorded measure of the time it took to render each frame. The following graphics cards consist of: Apple M1 Pro Chip Built-In GPU with 14 cores, Nvidia GeForce RTX 4060, ...

References

- [1] Yuri Bazlov, *Analysis: Lecture notes, chapter 2*, <https://personalpages.manchester.ac.uk/staff/yuri.bazlov/analysis/notes/ch2.pdf>, 2023, Accessed on June 12, 2025.
- [2] Rémi Coulon, Elisabetta A. Matsumoto, Henry Segerman, and Steve J. Trettel, *Ray-marching thurston geometries*, arXiv preprint arXiv:2010.15801 (2022), Accessed on June 2, 2025.
- [3] Joey de Vries, *Learnopengl — basic lighting*, <https://learnopengl.com/Lighting/Basic-Lighting>, 2017, Accessed on June 3, 2025.
- [4] Epic Games, *Volume texture — unreal engine 4.27 documentation*, https://dev.epicgames.com/documentation/en-us/unreal-engine/volume-texture?application_version=4.27, 2021, Accessed on June 17, 2025.
- [5] Diego Fonseca, *Raymarching application source code*, <https://github.com/Just-Feeshy/Raymarching-Application>, 2025, Accessed on June 2, 2025.
- [6] J. C. Hart, D. J. Sandin, and L. H. Kauffman, *Ray tracing deterministic 3-d fractals*, SIGGRAPH Computer Graphics **23** (1989), no. 3, 289–296.
- [7] John C. Hart, Daniel J. Sandin, and Louis H. Kauffman, *Sphere tracing: A geometric method for the antialiased ray tracing of implicit surfaces*, The Visual Computer, vol. 12, Springer, 1996, pp. 527–545.
- [8] Bastian Keinert, Martin Knecht, Hans-Peter Seidel, and Elmar Eisemann, *Enhanced sphere tracing*, Computer Graphics Forum **33** (2014), no. 4, 111–120.
- [9] Daniel Meister, Steffen Ogaki, Carsten Benthin, Michael J. Doyle, Michael Guthe, and Jiri Bittner, *A survey on bounding volume hierarchies for ray tracing*, Computer Graphics Forum **40** (2021), no. 1, 12–47.
- [10] Mehryar Mohri, *Semiring frameworks and algorithms for shortest-distance problems*, Journal of Automata, Languages and Combinatorics **7** (2002), no. 3, 321–350.
- [11] Maximilian Semrau and Jakub Sawicki, *Efficiency of 3d fractal generation through raymarching*, Proceedings of the 19th International Conference on Computer Graphics Theory and Applications (GRAPP 2024), SCITEPRESS – Science and Technology Publications, 2024, pp. 53–60.
- [12] Bernd Sturmfels, *Tropical mathematics*, The Mathematics Magazine **79** (2006), no. 3, 163–173.
- [13] Tenebryo, *Voxel raymarching*, <https://tenebryo.github.io/posts/2021-01-13-voxel-raymarching.html>, 2021, Accessed on June 8, 2025.